Plan maestro de pruebas de integración

Proyecto: NextGuard · Monitoreo IoT: ESP32 → API Python → PostgreSQL → Grafana
Entorno de ejecución: AWS Learner Lab, una EC2 con un contenedor Docker por servicio (api, db, grafana)
Versión: 1.1 · **Fecha:** 2026-10-10 · **Idioma:** español

---

1. Propósito y alcance

Este plan define cómo se comprobará que los componentes del sistema funcionan **juntos**: que una lectura tomada por un sensor conectado a un ESP32 llega por WiFi a una API en Python y queda guardada, sin pérdidas ni duplicados, en una base de datos SQL.

Dentro del alcance

- Sensor de temperatura y humedad (DHT11/DHT22) conectado al ESP32.
- Firmware del ESP32: muestreo, cola en RAM, reintentos, hora NTP y envío HTTP(S).
- API de ingesta (FastAPI) y su contrato HTTP.
- Base de datos PostgreSQL en un contenedor (y SQLite para las pruebas locales).
- Dashboards de Grafana sobre la base de datos (datasource y dashboard aprovisionados).
- Despliegue en AWS Learner Lab: una EC2 con Docker Compose, Security Group, Elastic IP.
- Pruebas no funcionales básicas (latencia, carga ligera, recuperación, ~3 h de operación continua) y seguridad básica.

Fuera del alcance (primera entrega)

- Sensores de material particulado y otras magnitudes: el diseño los admite (ver §2.3), pero no se configuran en el firmware todavía.
- Arduino sin WiFi propio: queda como caso opcional para una segunda etapa.
- Alta disponibilidad (varias instancias, balanceadores), alertas de Grafana, pruebas de penetración formales.

---

2. Sistema bajo prueba

2.1 Arquitectura

```
Sensor DHT ─► ESP32 ── WiFi ──► Internet ──► EC2 (docker compose)
              cola en RAM · NTP                ├─ api · FastAPI ──► db · PostgreSQL 16
              reintentos                       │   valida · clave     UNIQUE(dispositivo, seq)
                                               │   idempotencia       sin puertos publicados
                                               └─ grafana ──(grafana_ro, solo lectura)──► db
```

Diagramas detallados en `diagramas/index.html` (arquitectura, flujo de datos, secuencia de pruebas, despliegue AWS, niveles).

2.2 Contrato de la API

Método y ruta	Uso	Respuestas
`POST /api/v1/lecturas`	Una lectura	`201` creada · `200` duplicada (idempotente) · `401` clave · `413` demasiado grande · `422` inválida
`POST /api/v1/lecturas/batch`	Hasta 100 lecturas (cola acumulada)	`200` con `{insertadas, duplicadas}` · mismos errores
`GET /api/v1/lecturas`	Consulta por `device_id`, `desde`, `hasta`, `limit`	`200` · `401`
`GET /health`	Estado de API y conexión a la BD	`200` · `503` si la BD no responde

Autenticación: cabecera `X-API-Key`. Identificador de idempotencia: par (`device_id`, `seq`).

2.3 Modelo de datos

- `dispositivo` — un registro por ESP32 (`device_id` único).
- `lectura` — un envío; `UNIQUE(dispositivo_id, seq)`; guarda `ts_dispositivo` (NTP, puede ser nulo) y `ts_servidor`.
- `medicion` — formato largo (`magnitud`, `valor`, `unidad`): agregar PM2.5, PM10, CO_2, etc. **no requiere migrar el esquema** (caso `IT-DB-04`).

El DDL está en `db/schema.sql`. El rol `grafana_ro` (solo `SELECT`) se crea con `db/02-roles.sh`.

2.4 Decisiones de diseño que las pruebas deben confirmar

Decisión	Razón	Casos que la verifican
`seq` por dispositivo, persistido en NVS con salto de 20 al reiniciar	Un reintento no duplica y un reinicio no reutiliza números	IT-API-02, IT-DB-02, IT-E2E-03
Cola en RAM de 60 muestras, envío por lotes	Sobrevivir a cortes de red de hasta ~10 min	IT-API-20, IT-NET-03, IT-NFR-08
Formato largo para mediciones	Sensores futuros sin migraciones	IT-DB-04, IT-E2E-07

Decisión	Razón	Casos que la verifican
PostgreSQL sin puertos publicados; Grafana con usuario de solo lectura	Superficie de ataque mínima	IT-ENV-03, IT-ENV-04, IT-SEC-06
Un contenedor por servicio con `restart: unless-stopped` y volúmenes nombrados	El stack vuelve solo tras reiniciar la EC2 y no pierde datos	IT-ENV-05, IT-ENV-08, IT-NFR-07
Validación estricta (422) y API key	La API no confía en el dispositivo	IT-API-03…10, IT-SEC-01…05

---

3. Requisitos a verificar

ID	Requisito
RF-01	El sistema mide temperatura y humedad con el sensor conectado al ESP32.
RF-02	El ESP32 se conecta a la red WiFi y recupera la conexión tras una caída.
RF-03	El ESP32 envía cada lectura a la API en JSON, autenticado con API key.
RF-04	La API valida el contenido y lo persiste en la base SQL.
RF-05	No se pierden ni se duplican lecturas ante reintentos, cortes o reinicios.
RF-06	Las lecturas pueden consultarse por dispositivo y rango de tiempo.
RF-07	Se pueden agregar nuevas magnitudes sin cambiar el esquema.
RF-08	Un dashboard de Grafana muestra las lecturas almacenadas (temperatura, humedad, estado de los dispositivos).
RNF-01	Latencia de entrega (ESP32 → BD) con p95 menor a 2 s en condiciones normales.
RNF-02	El sistema se recupera solo tras caídas de red, API, BD o energía.
RNF-03	Seguridad básica: autenticación, validación de entradas, BD no expuesta, sin secretos en el código.
RNF-04	Trazabilidad temporal: hora del dispositivo (NTP) y hora del servidor en cada lectura.

La matriz completa requisito ↔ caso está en `02_casos_de_prueba.md` y en la hoja *Trazabilidad* de `casos_de_prueba.xlsx`.

---

4. Estrategia de pruebas

4.1 Niveles

Nivel	Qué integra	Método	Casos
N0 Entorno	EC2, Docker Compose, Security Group, Grafana	Manual + `smoke_env.sh`	IT-ENV-01…08
N1 Sensor ↔ ESP32	DHT y firmware	Manual (monitor serie)	IT-SEN-01…05

Nivel	Qué integra	Método	Casos
N2 ESP32 ↔ red y API	WiFi, NTP, contrato HTTP, reintentos	pytest + simulador + manual	IT-NET-01…05, IT-API-01…21
N3 API ↔ BD	Persistencia, restricciones, caída de BD	pytest (SQLite y PostgreSQL en contenedor)	IT-DB-01…11
N4 Extremo a extremo	Sensor real → BD → consulta	Manual + simulador	IT-E2E-01…07
N5 No funcionales	Latencia, carga, recuperación, operación continua	Simulador + observación	IT-NFR-01…09
N6 Seguridad básica	Autenticación, inyección, exposición	pytest + comprobación externa	IT-SEC-01…09

4.2 Enfoque: de abajo hacia arriba y sin depender del hardware

1. **Primero la API y la BD con datos simulados.** `simulador_sensor.py` reproduce el comportamiento del ESP32 (varios dispositivos, reenvíos duplicados, payloads inválidos, claves erróneas) y `pytest` fija el contrato. Así los defectos de servidor se descubren antes de conectar hardware.
2. **Después el ESP32 real** con el sensor simulado (`USE_DHT=0`) para aislar la red del sensor.
3. **Luego el sensor real** (`USE_DHT=1`) y la prueba extremo a extremo.
4. **Al final los fallos inducidos** (apagar router, detener la API, detener el contenedor db, cortar energía) y la operación continua.

4.3 Criterios de entrada y salida

Fase	Entrada (puede empezar si…)	Salida (termina cuando…)
N0	Laboratorio iniciado; EC2 permitida (t3.medium o t3.large)	`smoke_env.sh` sin fallas; IT-ENV-01…08 aprobados
N2/N3	N0 aprobado; API desplegada	Suite pytest 100 % en SQLite **y** en PostgreSQL; simulador con verificación OK
N1	Firmware compila; sensor cableado	IT-SEN-01…05 aprobados
N4	N1–N3 aprobados	IT-E2E-01…03 aprobados (0 faltantes, 0 duplicados)
N5/N6	N4 aprobado; ≥ 3,5 h de sesión para el soak	Ver criterios de aceptación (§8)

---

5. Entorno de pruebas

5.1 Recursos AWS (Learner Lab / sandbox)

Recurso	Configuración propuesta	Observaciones
EC2	Amazon Linux 2023, `t3.medium` (4 GB) como base; `t3.large` (8 GB) si se añaden servicios o para la carga IT-NFR-04. SG `nextguard-sg` (22 y 3000 solo «Mi IP», 8000 abierto al ESP32)	Ejecuta `docker compose` con los contenedores `api`, `db` y `grafana`; **Elastic IP** si está permitida. Límites de memoria en compose: db 1 GB, api 512 MB, grafana 512 MB

Recurso	Configuración propuesta	Observaciones
PostgreSQL	Contenedor `postgres:16`, volumen `pgdata`, sin puertos publicados	Base `sensores`, usuario `sensores_app` y rol `grafana_ro` (solo lectura)
Security Group	`nextguard-sg`	Sin regla para el puerto 5432
IAM	`LabRole` / `LabInstanceProfile`	No se pueden crear roles propios

La guía paso a paso está en `docs/deploy_ec2.md`.

5.2 Restricciones del laboratorio a tener presentes

> Verificar en la cuenta real: los servicios habilitados y las cuotas pueden variar entre laboratorios y sandboxes.

- **Sesión de ~4 h:** al vencer, las EC2 se detienen. La prueba de operación continua (3 h) debe iniciarse con al menos 3,5 h restantes.
- **IP pública variable** si no hay Elastic IP: cambiar `API_HOST` en `config.h` y volver a cargar el firmware en cada sesión.
- **Presupuesto limitado:** detener la EC2 al cerrar cada sesión.
- **Región habitual:** `us-east-1` o `us-west-2`.

5.3 Entorno local (PC)

Python 3.11+ con `scripts/requirements-dev.txt`; SQLite. Permite ejecutar toda la suite y el simulador sin AWS.

5.4 Hardware

ESP32 (DevKit o similar) con cable USB de datos de calidad, sensor DHT11 o DHT22 (con resistencia pull-up de 10 kΩ si el módulo no la trae), protoboard y cables, router 2.4 GHz con WPA2, termómetro/higrómetro de referencia, segundo dispositivo para probar el hotspot (opcional).

5.5 Conexión WiFi (lección de la etapa de diagnóstico)

Durante la puesta en marcha el ESP32 no conectaba y la causa era una contraseña mal escrita. El diagnóstico con `wifi_diagnostico.ino` (escaneo + motivo de desconexión) permitió descartar banda, canal y señal. El escaneo también mostró **dos puntos de acceso con el mismo SSID** (posible repetidor); el firmware ya elige el de mejor señal. Este mismo mecanismo se reutiliza en los casos IT-NET-01 y IT-NET-02.

---

6. Herramientas y scripts

Elemento	Ubicación	Función
API FastAPI	`api/` (con `Dockerfile`)	Sistema bajo prueba
Stack Docker	`deploy/docker-compose.yml`	Servicios `api`, `db` y `grafana` en una EC2
Dashboards	`deploy/grafana/`	Datasource PostgreSQL y dashboard «NextGuard · Monitoreo IoT» aprovisionados
Esquema SQL	`db/schema.sql`, `db/02-roles.sh`	DDL PostgreSQL y rol de solo lectura
Suite pytest	`tests/`	Contrato, BD, seguridad, carga ligera (`pytest`)
Simulador	`scripts/simulador_sensor.py`	Genera tráfico; `--verify` compara lo enviado con lo guardado
Prueba de humo	`scripts/smoke_env.sh`	Verifica API, Grafana y que PostgreSQL no sea accesible desde fuera
Firmware	`firmware/esp32_envio/`	ESP32 con cola, NTP y reintentos
Diagnóstico WiFi	`firmware/wifi_diagnostico/`	Escaneo y motivo de desconexión
Generador de documentos	`scripts/generar_documentos.py`	Crea .md, .csv y .xlsx desde `casos_de_prueba.py`

Comandos frecuentes:

```
# Suite completa en local (SQLite)
pytest
# Stack completo en la EC2 (o en tu PC con Docker)
cd deploy && cp .env.example .env && docker compose up -d --build
# Suite contra PostgreSQL (BD de pruebas vacía; publicar 5432 solo temporalmente)
TEST_DATABASE_URL=postgresql+psycopg://sensores_app:clave@localhost:5432/sensores_test pytest -m
"not carga"
# Tráfico simulado con verificación
python simulador_sensor.py --url http://<IP>:8000 --api-key <clave> --devices 3 --count 50 --
dup-rate 0.1 --verify
```

---

7. Plan de ejecución

Cada sesión de laboratorio dura ~4 h; se planifican **sesiones cortas y acotadas** para no depender de recursos que expiran.

Sesión	Objetivo	Casos	Duración estimada
S0 *(PC, sin AWS)*	Suite pytest y simulador contra SQLite local	IT-API-01…17, IT-DB-01…09, IT-SEC-01…05, IT-NFR-01/02	30 min
S1	Crear SG y EC2; instalar Docker; `docker compose up`; `smoke_env.sh`; revisar Grafana	IT-ENV-01…08, IT-SEC-06	2–3 h
S2	Suite pytest contra PostgreSQL; simulador desde fuera de AWS	IT-DB-10, IT-NFR-03, IT-E2E-04/06/07	1–2 h

Sesión	Objetivo	Casos	Duración estimada
S3	Firmware con sensor simulado y luego DHT real	IT-SEN-01…05, IT-NET-01/02/04, IT-API-18/19	2 h
S4	Prueba extremo a extremo con ESP32 real	IT-E2E-01…03/05	1 h
S5	Fallos inducidos y recuperación	IT-NET-03, IT-API-20/21, IT-DB-11, IT-NFR-06/07/08	2 h
S6	Carga y seguridad	IT-NFR-04, IT-NET-05, IT-SEC-07/08/09	1–2 h
S7	Operación continua de 3 h y cierre	IT-NFR-05, IT-NFR-09	3,5 h

Al terminar cada sesión: guardar evidencias, actualizar `casos_de_prueba.xlsx`, detener la EC2.

---

8. Criterios de aceptación

El sistema se considera aceptado para la primera entrega si se cumple **todo** lo siguiente:

#	Criterio	Métrica
1	Casos de prioridad Alta	100 % aprobados
2	Casos de prioridad Media	≥ 90 % aprobados; los restantes con defecto registrado y plan
3	Integridad de datos	0 lecturas duplicadas y 0 faltantes en IT-E2E-01 y en la operación continua (tolerancia: ≤ 1 % de pérdida solo en la operación continua)
4	Latencia	p95 < 2 s medida desde fuera de AWS (IT-NFR-03)
5	Recuperación	El sistema retoma el envío por sí solo tras caída de router, API y contenedor db (IT-NET-03, IT-API-20, IT-DB-11)
6	Seguridad	PostgreSQL no accesible desde Internet; sin secretos en el código; 0 inyecciones SQL exitosas
7	Defectos	0 abiertos de severidad 1 o 2
8	Evidencia	Cada caso ejecutado con resultado real y evidencia en la matriz

8.1 Severidad de defectos

Severidad	Definición	Ejemplo
1 – Crítica	Pérdida o corrupción de datos, exposición de secretos	Lecturas duplicadas; puerto 5432 publicado
2 – Alta	Función principal inutilizable	La API responde 500 a una lectura válida
3 – Media	Función degradada con alternativa	Reintento sin backoff
4 – Baja	Cosmético o de documentación	Mensaje poco claro

Cada defecto se registra con: ID de caso, pasos, resultado esperado vs real, evidencia (captura del monitor serie, log de `docker compose logs`, consulta SQL), severidad y estado.

9. Riesgos

ID	Riesgo	Prob.	Impacto	Mitigación
R-01	La IP pública de la EC2 cambia al reiniciar y el ESP32 deja de enviar	Alta	Alto	Elastic IP; si no es posible, actualizar `config.h` y documentar el procedimiento
R-02	La sesión del laboratorio vence durante la operación continua	Media	Alto	Iniciar con ≥ 3,5 h restantes; registrar la hora de inicio; reanudar y comparar por `seq`
R-03	Presupuesto del laboratorio agotado	Media	Alto	Detener la EC2 al cerrar; usar `t3.medium` y subir a `t3.large` solo si hace falta
R-04	El tipo de instancia (`t3.medium` / `t3.large`) o Docker no están disponibles en el sandbox	Media	Alto	Verificar en S1; alternativa: `t3.small` con límites de memoria menores y sin Grafana durante la carga
R-13	La EC2 queda sin memoria al correr api, db y grafana juntos (carga IT-NFR-04)	Media	Medio	`mem_limit` por contenedor; observar con `docker stats`; subir a `t3.large`
R-14	`docker compose down -v` borra los datos	Baja	Alto	Documentado en `docs/deploy_ec2.md`: usar `down` sin `-v`; exportar resultados antes de cerrar
R-05	Lecturas `NaN` o poco precisas del DHT (especialmente DHT11)	Media	Medio	El firmware descarta `NaN`; comparar con instrumento de referencia; registrar la tolerancia del sensor
R-06	Caídas de tensión (cable/puerto USB) provocan fallos de WiFi	Media	Medio	Cable de datos corto y de calidad; puerto directo; alimentación externa; `USE_LOW_TX_POWER` en el diagnóstico
R-07	Cobertura 2.4 GHz insuficiente o repetidor con el mismo SSID	Media	Medio	Firmware con selección por mejor señal; medir con IT-NET-05
R-08	Tráfico HTTP sin cifrar (API key visible en la red)	Alta	Medio	Riesgo aceptado en el laboratorio; fase 2 con HTTPS; documentar en IT-SEC-09
R-09	Corte de energía pierde la cola en RAM (hasta 60 muestras)	Media	Bajo	Documentar la pérdida máxima; opción futura: persistir la cola en flash
R-10	Reloj sin NTP genera lecturas sin `ts_dispositivo`	Media	Bajo	La API acepta `null` y usa `ts_servidor`; IT-NET-04
R-11	Secretos expuestos (contraseña WiFi, API key, credenciales de BD, claves de Grafana)	Media	Alto	`config.h` y `.env` fuera del repositorio (el repo es público); rotar la clave WiFi que se compartió durante el diagnóstico; IT-SEC-07
R-12	El salto de `seq` tras un reinicio se interpreta como pérdida	Baja	Bajo	Documentado: el salto es de hasta 20 por diseño; los análisis de integridad distinguen reinicios

10. Estado de verificación al momento de emitir este plan

Para no confundir lo **diseñado** con lo **comprobado**:

Elemento	Estado
Suite pytest (33 pruebas) contra SQLite local	✅ Ejecutada; todas pasan

Elemento	Estado
API en `uvicorn` + simulador con `--verify` (varios dispositivos, duplicados, casos inválidos, 20 dispositivos en paralelo)	✅ Ejecutado en local; sin faltantes ni duplicados
`smoke_env.sh` contra la API local	✅ Ejecutado
Matriz `.xlsx` (fórmulas de resumen)	✅ Recalculada con LibreOffice
Hallazgo durante las pruebas	Se detectó y corrigió un defecto: un valor `NaN` provocaba error 500 en lugar de 422 (caso IT-API-07)
Stack `docker compose` (api + PostgreSQL 16 + Grafana) en local	✅ Levantado con Docker 29: `/health` ok, simulador con `--verify` OK contra PostgreSQL y consultas de los 8 paneles de Grafana devuelven 200
Suite pytest completa contra PostgreSQL (IT-DB-10)	⏳ Pendiente; la API sí se probó contra PostgreSQL con el simulador
Firmware `esp32_envio.ino`	⏳ **Sin compilar** en el entorno de desarrollo; compilar y probar en el Arduino IDE
Despliegue en la EC2 y pruebas con hardware	⏳ Pendiente (S1–S7)

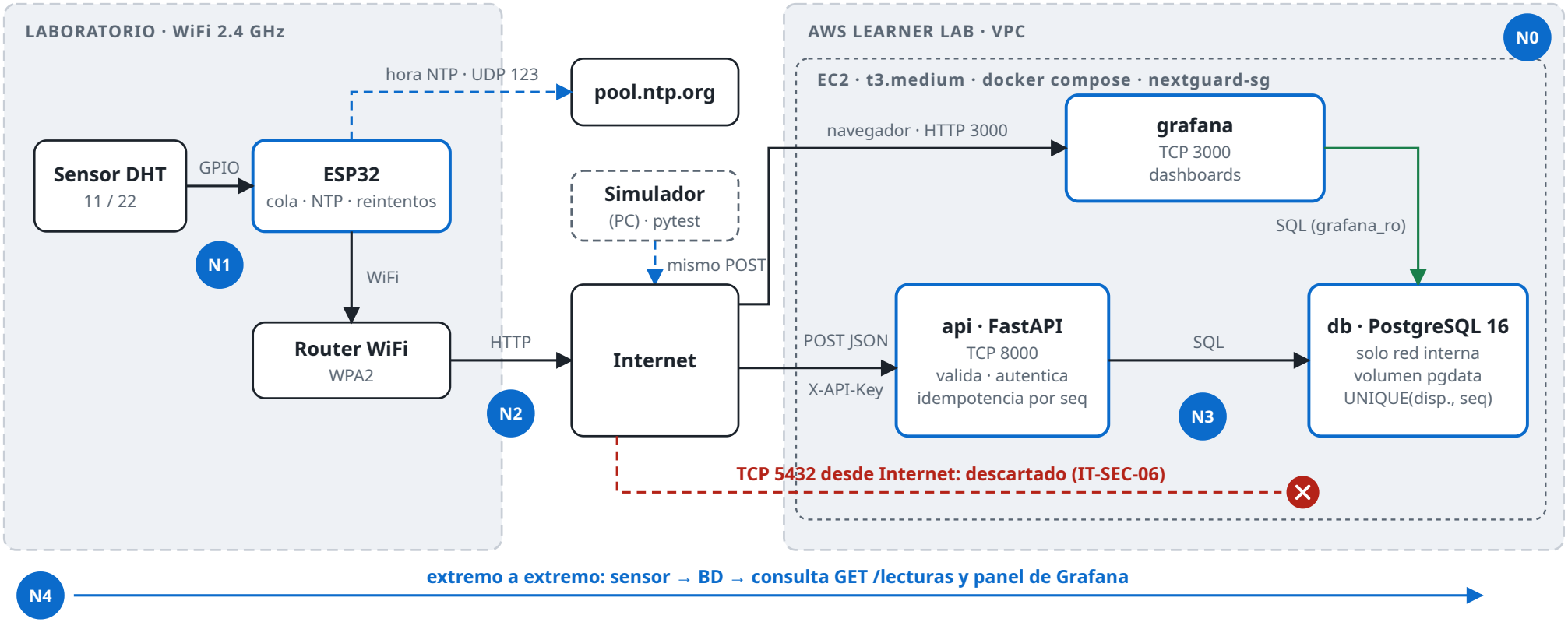
11. Entregables del paquete

```
NextGuard-monitoreo-IoT/
├── README.md                    índice y cómo empezar
├── api/                         API FastAPI + Dockerfile
├── db/                          schema.sql y rol de solo lectura para Grafana
├── deploy/                      docker-compose.yml, .env.example y dashboards de Grafana
├── firmware/                    esp32_envio y wifi_diagnostico (config.h.example)
├── tests/                       pytest
├── scripts/                     simulador, humo y generadores de documentos
└── docs/                        este plan (.md/.pdf), casos de prueba, deploy_ec2.md y
diagramas/ (HTML + PDF)
```

12. Aprobaciones

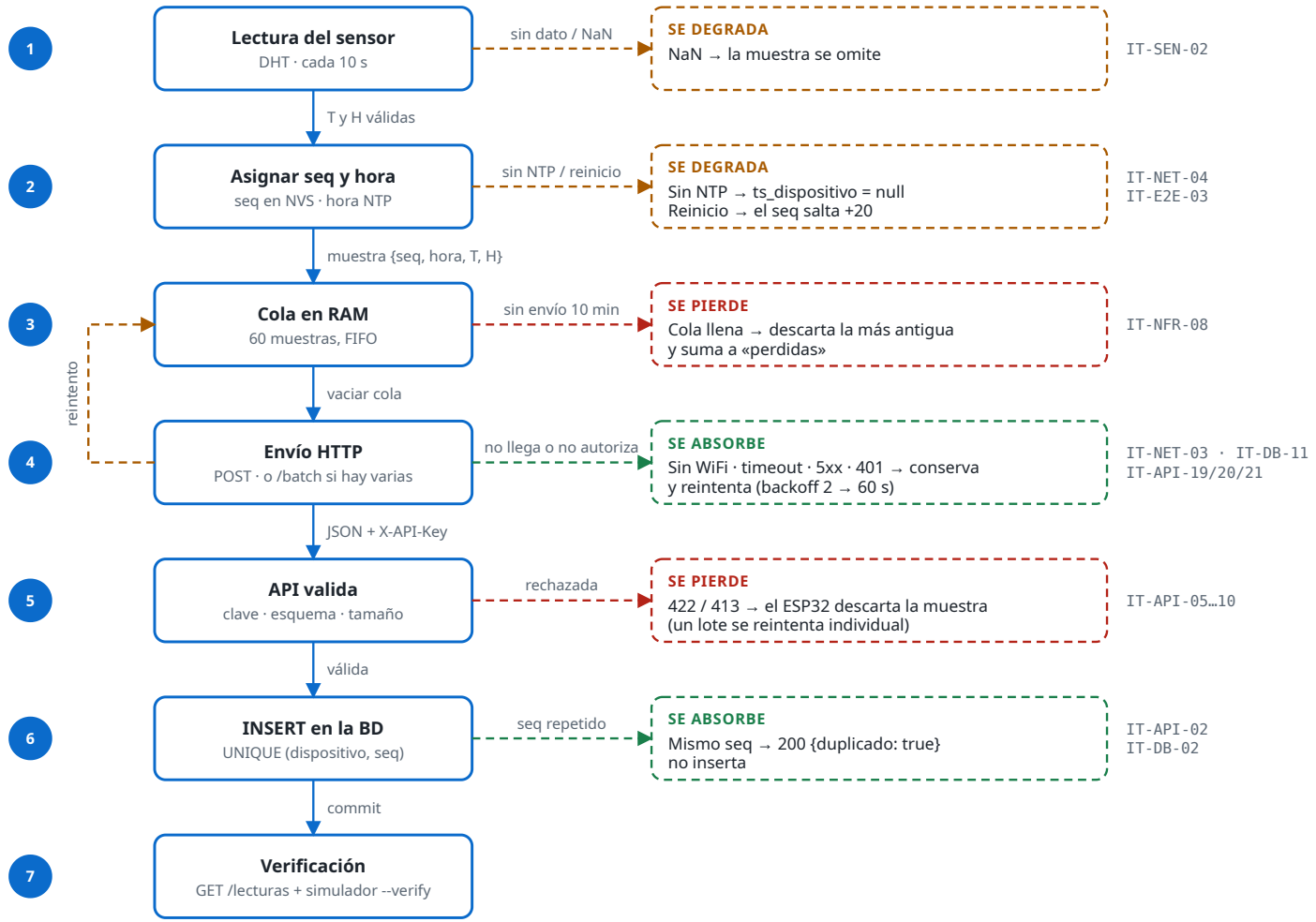
Rol	Nombre	Fecha	Firma
Responsable del proyecto			
Docente / revisor			

Anexo A.1 · Arquitectura bajo prueba



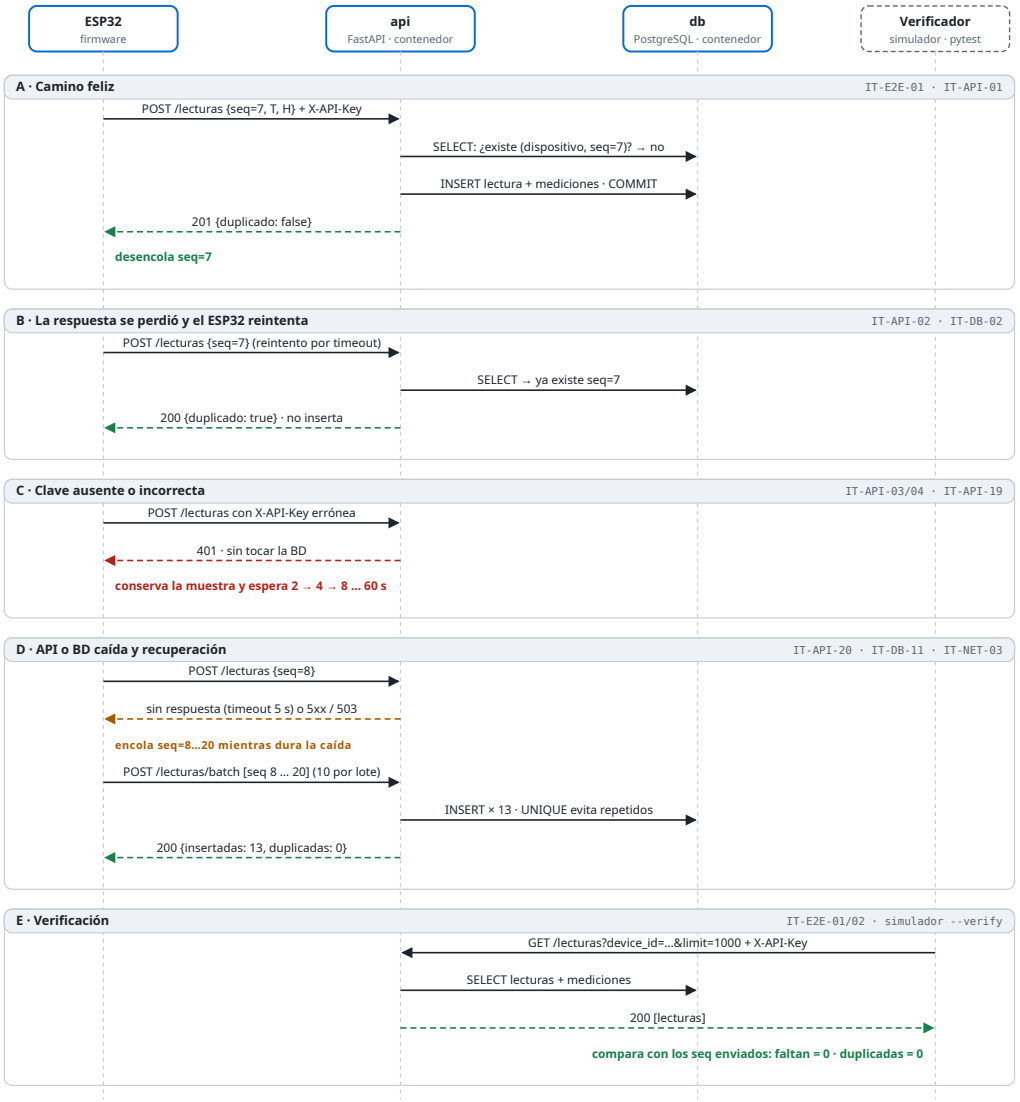
El ESP32 solo conoce a la API; la API es el único componente que escribe en PostgreSQL y Grafana solo lo lee, con un usuario de solo lectura. Los tres servicios son contenedores de una misma EC2. El simulador reemplaza al ESP32 con el mismo POST, y la línea roja muestra el camino que **debe fallar**: PostgreSQL no publica su puerto.

Anexo A.2 · Recorrido de una lectura



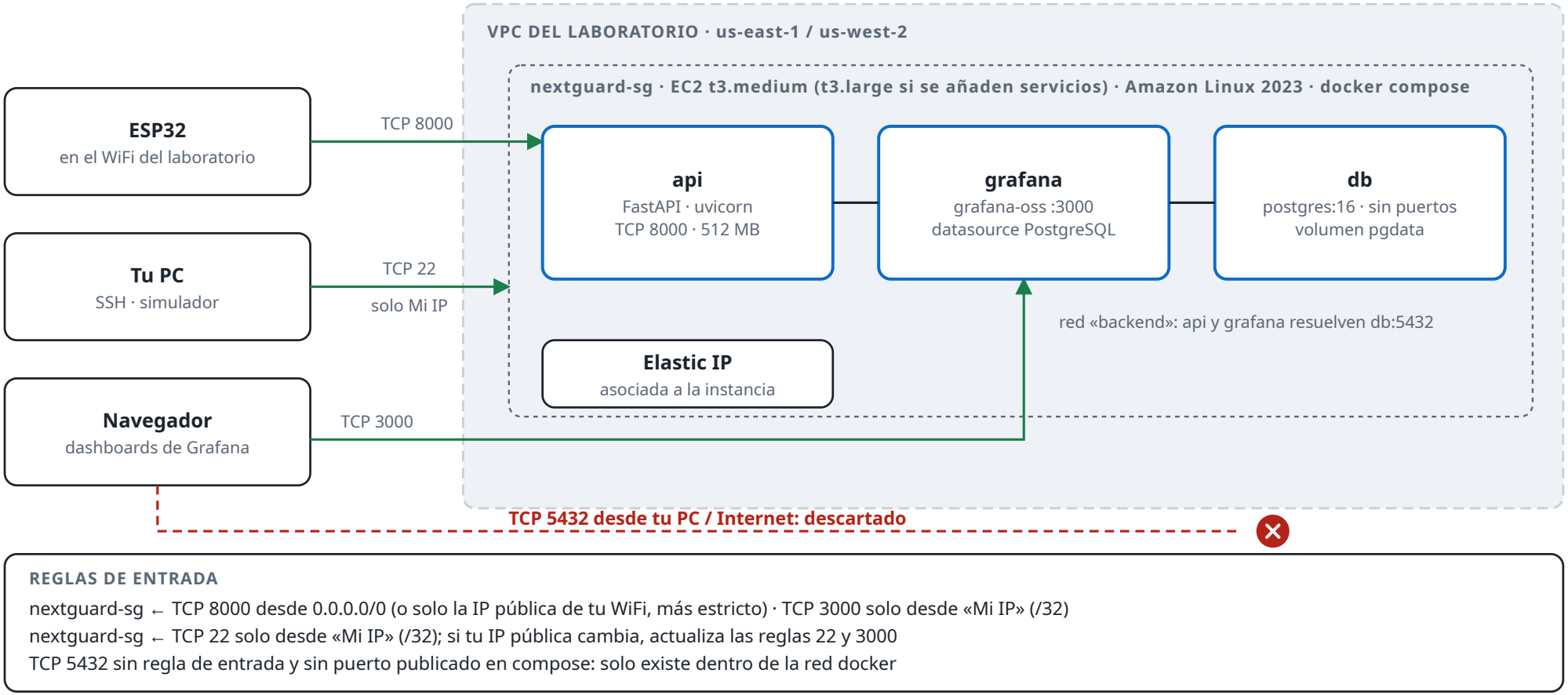
Cada recuadro punteado es un fallo inducido en las pruebas. **Se absorbe**: no se pierde nada. **Se degrada**: la lectura llega con menos información. **Se pierde**: una muestra se descarta y queda contada.

Anexo A.3 · Secuencia de las pruebas



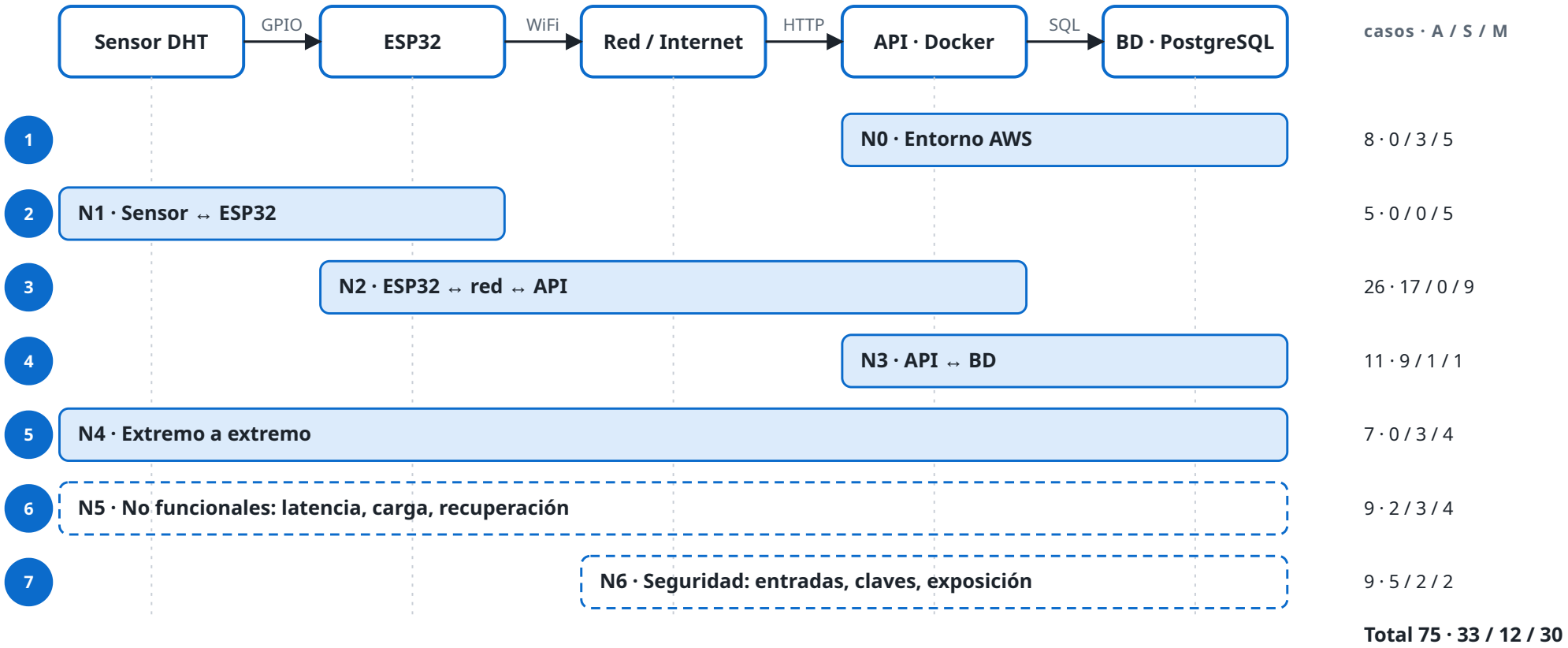
Las líneas discontinuas son respuestas. Verde = éxito, ámbar = degradación recuperable, rojo = rechazo. El escenario D es el que justifica la cola en el ESP32 y el endpoint `/batch`.

Anexo A.4 · Despliegue en AWS



Flechas verdes = conexiones permitidas. La línea roja discontinua es la que debe rechazarse: PostgreSQL no publica su puerto y el Security Group no tiene regla para 5432.

Anexo A.5 · Niveles de prueba



Orden de ejecución: 1 → 7. A = automatizado (pytest) · S = semi-automático · M = manual

Cada barra cubre los componentes que ese nivel integra. Los niveles se ejecutan de arriba abajo: primero el entorno, luego cada unión por separado y solo después el camino completo. Las barras punteadas (N5, N6) no añaden una unión nueva: someten al conjunto a fallos, carga y ataques.